

NonLinear

Machine Learning Implementation for Network Intrusion Detection

> Essential ML process for Intrusion Detection

python 3.7.13 scikit-learn 1.0.2

numpy 1.19.5 pandas 1.3.5

Import the main libraries

```
In [1]: class="keyword">import numpy
class="keyword">import pandas

class="keyword">from time class="keyword">import time

class="keyword">import os
data_path = class="string">'../data' class="string">"comment">#
Updated for Assignment folder structure
```

import the local library

```
In [2]: class="string">"comment"># add parent folder path where lib folder is
class="keyword">import sys
if class="string">".." not in sys.path:class="keyword">import sys;
sys.path.insert(class="number">0, class="string">'..')
```

```
In [3]: class="keyword">from mylib class="keyword">import show_labels_dist,
show_metrics, bias_var_metrics
```

```
/usr/local/lib/python3.12/dist-
packages/matplotlib/projections/__init__.py:63: UserWarning: Unable to
import Axes3D. This may be due to multiple versions of Matplotlib being
installed (e.g. as a system package and as a pip package). As a result, the
3D projection is not available.
  warnings.warn("Unable to import Axes3D. This may be due to multiple
versions of "
```

Import the Dataset

```
In [4]: class=class="string">"comment"># Using boosted Train and preprocessed Test

data_file = os.path.join(data_path, class="string">'NSL_boosted-
class="number">2.csv')
train_df = pandas.read_csv(data_file)
print(class="string">'Train Dataset: {} rows, {}
columns'.format(train_df.shape[class="number">0],
train_df.shape[class="number">1]))

data_file = os.path.join(data_path, class="string">'NSL_ppTest.csv')
test_df = pandas.read_csv(data_file)
print(class="string">'Test Dataset: {} rows, {}
columns'.format(test_df.shape[class="number">0],
test_df.shape[class="number">1]))
```

```
Train Dataset: 63280 rows, 43 columns
Test Dataset: 22544 rows, 43 columns
```

*

Data Preparation and EDA (unique to this dataset)

- _Check column names of numeric attributes_

```
In [5]: trnn = train_df.select_dtypes(include=
[class="string">'float64',class="string">'int64']).columns
tstn = test_df.select_dtypes(include=
[class="string">'float64',class="string">'int64']).columns
trndif = numpy.setdiff1d(trnn, tstn)
tstdif = numpy.setdiff1d(tstn, trnn)

print(class="string">"Numeric features in the train_set that are not in the
test_set: ",end=class="string">'')
if len(trndif) > class="number">0:
    print(class="string">'\\n',trndif)
else:
    print(class="string">'None')

print(class="string">"Numeric features in the test_set that are not in the
train_set: ",end=class="string">'')
if len(tstdif) > class="number">0:
    print(class="string">'\\n',tstdif)
else:
    print(class="string">'None')

print()
class=class="string">"comment"># correct any differences here
```

```
Numeric features in the train_set that are not in the test_set: None
Numeric features in the test_set that are not in the train_set: None
```

- _Check column names of categorical attributes_

```
In [6]: trnn = train_df.select_dtypes(include=[class="string">'object'] ).columns
tstn = test_df.select_dtypes(include=[class="string">'object'] ).columns
trndif = numpy.setdiff1d(trnn, tstn)
tstdif = numpy.setdiff1d(tstn, trnn)

print(class="string">"Categorical features in the train_set that are not in
the test_set: ",end=class="string">'')
if len(trndif) > class="number">0:
    print(class="string">'\\n',trndif)
else:
    print(class="string">'None')

print(class="string">"Categorical features in the test_set that are not in
the train_set: ",end=class="string">'')
if len(tstdif) > class="number">0:
    print(class="string">'\\n\\t',tstdif)
else:
    print(class="string">'None')

print()
class=class="string">"comment"># correct any differences here
```

```
Categorical features in the train_set that are not in the test_set: None
Categorical features in the test_set that are not in the train_set: None
```

- _Check for missing values_

```

In [7]: cnt=class="number">0
print(class="string">'Missing Values - Train Set')
for col in train_df.columns:
class=class="string">"comment">#      print(col, class="string">' ::> ',
len(combined_df[col].unique()))
    nnul = pandas.notnull(train_df[col])
    if (len(nnul)!=len(train_df)):
        cnt=cnt+class="number">1
        print(class="string">'\\t',col,class="string">':',(len(test_df)-
len(nnul)),class="string">'null values')
print(class="string">'Total',cnt,class="string">'features with null
values')

cnt=class="number">0
print(class="string">'Missing Values - Test Set')
for col in test_df.columns:
class=class="string">"comment">#      print(col, class="string">' ::> ',
len(combined_df[col].unique()))
    nnul = pandas.notnull(test_df[col])
    if (len(nnul)!=len(test_df)):
        cnt=cnt+class="number">1
        print(class="string">'\\t',col,class="string">':',(len(test_df)-
len(nnul)),class="string">'null values')
print(class="string">'Total',cnt,class="string">'features with null
values')

class=class="string">"comment"># address missing values here

```

```

Missing Values - Train Set
Total 0 features with null values
Missing Values - Test Set
Total 0 features with null values

```

- _Quick visual check of unique values, deal with unique identifiers_

```
In [8]: class=class="string">"comment"># Identify columns with only one value
class=class="string">"comment"># or with number of unique values == number
of rows
n_eq_one = []
n_eq_all = []

print(class="string">'Unique value count:
Train(' ,train_df.shape[class="number">0],class="string">'rows ) ~
Test(' ,test_df.shape[class="number">0],class="string">'rows )')
for col in train_df.columns:
    lctrn = len(train_df[col].unique())
    lctst = len(test_df[col].unique())
    print(col, class="string">' ::> ' , lctrn, class="string">' ~ ' , lctst)
    if (lctrn == class="number">1) and (lctrn == lctst):
        n_eq_one.append(train_df[col].name)
    if lctrn == train_df.shape[class="number">0]:
        n_eq_all.append(train_df[col].name)
```

```
Unique value count: Train ( 63280 rows ) ~ Test( 22544 rows )
duration ::> 1657 ~ 624
protocol_type ::> 3 ~ 3
service ::> 70 ~ 64
flag ::> 11 ~ 11
src_bytes ::> 2553 ~ 1149
dst_bytes ::> 6633 ~ 3650
land ::> 2 ~ 2
wrong_fragment ::> 3 ~ 3
urgent ::> 3 ~ 4
hot ::> 24 ~ 16
num_failed_logins ::> 6 ~ 5
logged_in ::> 2 ~ 2
num_compromised ::> 49 ~ 23
root_shell ::> 2 ~ 2
su_attempted ::> 3 ~ 3
num_root ::> 44 ~ 20
num_file_creations ::> 28 ~ 9
num_shells ::> 3 ~ 4
num_access_files ::> 9 ~ 5
num_outbound_cmds ::> 1 ~ 1
is_host_login ::> 2 ~ 2
is_guest_login ::> 2 ~ 2
count ::> 511 ~ 495
srv_count ::> 489 ~ 457
serror_rate ::> 87 ~ 88
srv_serror_rate ::> 75 ~ 82
rerror_rate ::> 81 ~ 90
srv_rerror_rate ::> 63 ~ 93
same_srv_rate ::> 101 ~ 75
diff_srv_rate ::> 88 ~ 99
srv_diff_host_rate ::> 61 ~ 84
dst_host_count ::> 256 ~ 256
dst_host_srv_count ::> 256 ~ 256
dst_host_same_srv_rate ::> 101 ~ 101
dst_host_diff_srv_rate ::> 101 ~ 101
dst_host_same_src_port_rate ::> 101 ~ 101
dst_host_srv_diff_host_rate ::> 67 ~ 58
dst_host_serror_rate ::> 101 ~ 99
```

```
dst_host_srv_error_rate ::> 100 ~ 101
dst_host_rerror_rate ::> 101 ~ 101
dst_host_srv_rerror_rate ::> 101 ~ 100
label ::> 40 ~ 38
atakat ::> 5 ~ 5
```

```
In [9]: class=class="string">"comment"># Drop columns with only one value
if len(n_eq_one) > class="number">0:
    print(class="string">'Dropping single-valued features')
    print(n_eq_one)
    train_df.drop(n_eq_one, axis=class="number">1, inplace=True)
    test_df.drop(n_eq_one, axis=class="number">1, inplace=True)

class=class="string">"comment"># Drop or bin columns with number of unique
values == number of rows
if len(n_eq_all) > class="number">0:
    print(class="string">'Dropping unique identifiers')
    print(n_eq_all)
    train_df.drop(n_eq_all, axis=class="number">1, inplace=True)
    test_df.drop(n_eq_all, axis=class="number">1, inplace=True)

class=class="string">"comment"># continue with feature selection / feature
engineering
```

```
Dropping single-valued features
['num_outbound_cmds']
```

- _Check categorical feature values:

differences will be resolved by one-hot encoding the combined test and train sets_

```

In [10]: trnn = train_df.select_dtypes(include=[class="string">'object']).columns
for col in trnn:
    tr = train_df[col].unique()
    ts = test_df[col].unique()
    trd = numpy.setdiff1d(tr, ts)
    tsd = numpy.setdiff1d(ts, tr)

    print(col,class="string">'::> ')
    print(class="string">"\tUnique text values in the train_set that are
not in the test_set: ",end=class="string">'')
    if len(trd) > class="number">0:
        print(class="string">'\\n\\t',trd)
    else:
        print(class="string">'None')

    print(class="string">"\tUnique text values in the test_set that are
not in the train_set: ",end=class="string">'')
    if len(tsd) > class="number">0:
        print(class="string">'\\n\\t',tsd)
    else:
        print(class="string">'None')

```

```

protocol_type ::>
    Unique text values in the train_set that are not in the test_set:
None
    Unique text values in the test_set that are not in the train_set:
None
service ::>
    Unique text values in the train_set that are not in the test_set:
['aol' 'harvest' 'http_2784' 'http_8001' 'red_i' 'urh_i']
    Unique text values in the test_set that are not in the train_set:
None
flag ::>
    Unique text values in the train_set that are not in the test_set:
None
    Unique text values in the test_set that are not in the train_set:
None
label ::>
    Unique text values in the train_set that are not in the test_set:
['spy' 'warezclient']
    Unique text values in the test_set that are not in the train_set:
None
atakcat ::>
    Unique text values in the train_set that are not in the test_set:
None
    Unique text values in the test_set that are not in the train_set:
None

```

- _Combine for processing classification target and text features_

```
In [11]: combined_df = pandas.concat([train_df, test_df])
print(class="string">'Combined Dataset: {} rows, {} columns'.format(
    combined_df.shape[class="number">0],
    combined_df.shape[class="number">1]))
```

```
Combined Dataset: 85824 rows, 42 columns
```

- _Classification Target feature:_

two columns of labels are available

* Two-class: Reduce the detailed attack labels to 'normal' or 'attack'

* Multiclass: Use the category labels (atakat)

```
In [12]: combined_df[class="string">'label'].value_counts()
```

```
Out[12]: label
normal          43383
neptune         25264
satan           2552
smurf           1988
ipsweep         1941
portsweep       1623
guess_passwd    1257
mscan           1046
warezmaster     954
back            837
nmap            820
apache2         774
processtable    719
teardrop        458
warezclient     445
snmpguess       364
saint           350
mailbomb        322
snmpgetattack   196
httptunnel      146
pod             142
buffer_overflow 35
named           25
ps              22
multihop        21
sendmail        21
xterm           20
rootkit         18
land            16
xlock           14
xsnoop          8
ftp_write       7
imap            6
loadmodule      6
phf             6
perl            5
worm            4
sqlattack       4
udpstorm        4
spy             1
Name: count, dtype: int64
```

```
In [13]: combined_df[class="string">'atakat'].value_counts()
```

```
Out[13]: atakat
benign    43383
dos       30524
probe     8332
r2l       3329
u2r       256
Name: count, dtype: int64
```

```
In [14]: class=class="string">"comment"># Set the classification target
twoclass = True      class=class="string">"comment"># True or False
```

```
In [15]: if twoclass:
class=class="string">"comment"># Two-class: Reduce the detailed attack
labels to class="string">'normal' or class="string">'attack'
class=class="string">"comment"># new single column data structure is a
[series]
    labels_df = combined_df[class="string">'label'].copy()
    labels_df[labels_df != class="string">'normal'] =
class="string">'attack'
else:
class=class="string">"comment"># Multiclass: Use the category
labels(atakat)
class=class="string">"comment"># new single column data structure is a
[[dataframe]]
class=class="string">"comment"># rename the column and convert to a series
for later
    labels_df = combined_df[[class="string">'atakat']].copy()
    labels_df.rename(columns=
{class="string">'atakat':class="string">'label'}, inplace=True)
    labels_df = labels_df.squeeze(class="string">'columns')

class=class="string">"comment"># drop target features
combined_df.drop([class="string">'label'], axis=class="number">1,
inplace=True)
combined_df.drop([class="string">'atakat'], axis=class="number">1,
inplace=True)
```

- One-Hot Encoding the categorical (text) features

```
In [16]: class=class="string">"comment"># put the names into a python list - for
pandas.get_dummies()
categori = combined_df.select_dtypes(include=
[class="string">'object']).columns
category_cols = categori.tolist()
print(category_cols)
```

```
['protocol_type', 'service', 'flag']
```

```
In [17]: class=class="string">"comment"># generate a sorted list of unique values
of categorical features
class=class="string">"comment"># we will get a new column for each one
with get_dummies()

for col in categori:
    ul = numpy.sort(list(combined_df[col].unique()))
    print(col, class="string">' ::> ', ul)
    print()
```

```
protocol_type ::> ['icmp' 'tcp' 'udp']

service ::> ['IRC' 'X11' 'Z39_50' 'aol' 'auth' 'bgp' 'courier' 'csnet_ns'
'ctf'
'daytime' 'discard' 'domain' 'domain_u' 'echo' 'eco_i' 'ecr_i' 'efs'
'exec' 'finger' 'ftp' 'ftp_data' 'gopher' 'harvest' 'hostnames' 'http'
'http_2784' 'http_443' 'http_8001' 'imap4' 'iso_tsap' 'klogin' 'kshell'
'ldap' 'link' 'login' 'mtp' 'name' 'netbios_dgm' 'netbios_ns'
'netbios_ssn' 'netstat' 'nnsdp' 'nntp' 'ntp_u' 'other' 'pm_dump' 'pop_2'
'pop_3' 'printer' 'private' 'red_i' 'remote_job' 'rje' 'shell' 'smtp'
'sql_net' 'ssh' 'sunrpc' 'supdup' 'systat' 'telnet' 'tftp_u' 'tim_i'
'time' 'urh_i' 'urp_i' 'uucp' 'uucp_path' 'vmnet' 'whois']

flag ::> ['OTH' 'REJ' 'RSTO' 'RSTOS0' 'RSTR' 'S0' 'S1' 'S2' 'S3' 'SF'
'SH']
```

```
In [18]: class=class="string">"comment"># Apply to the list of Categorical
columns(text fields)
features_df = pandas.get_dummies(combined_df, columns=category_cols)
features_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 85824 entries, 0 to 22543
Columns: 121 entries, duration to flag_SH
dtypes: bool(84), float64(15), int64(22)
memory usage: 31.8 MB
```

```
In [19]: class=class="string">"comment"># generate a list of numeric columns for
scaling - After test // train split
numeri = combined_df.select_dtypes(include=
[class="string">'float64',class="string">'int64']).columns
print(numeri.to_list())
```

```
['duration', 'src_bytes', 'dst_bytes', 'land', 'wrong_fragment', 'urgent',
'hot', 'num_failed_logins', 'logged_in', 'num_compromised', 'root_shell',
'su_attempted', 'num_root', 'num_file_creations', 'num_shells',
'num_access_files', 'is_host_login', 'is_guest_login', 'count',
'srv_count', 'error_rate', 'srv_error_rate', 'rerror_rate',
'srv_rerror_rate', 'same_srv_rate', 'diff_srv_rate', 'srv_diff_host_rate',
'dst_host_count', 'dst_host_srv_count', 'dst_host_same_srv_rate',
'dst_host_diff_srv_rate', 'dst_host_same_src_port_rate',
'dst_host_srv_diff_host_rate', 'dst_host_error_rate',
'dst_host_srv_error_rate', 'dst_host_rerror_rate',
'dst_host_srv_rerror_rate']
```

*

Create Test // Train Datasets

> Normally we split the dataset into train 70 % // test 30 % like this

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test =
```

```
train_test_split(features_df, labels_df,
```

```
test_size=0.3, stratify=labels_df, random_state=42)
```

```

In [20]: class=class="string">"comment"># Restore the train // test split: slice
class="number">1 Dataframe into class="number">2
features_train = features_df.iloc[:len(train_df),:].copy()
class=class="string">"comment"># X_train
features_train.reset_index(inplace=True, drop=True)
class=class="string">"comment"># pandas has a lot of rules about returning
a class="string">'view' vs. a copy class="keyword">from slice
class=class="string">"comment"># so we force it to create a new dataframe
[avoiding SettingWithCopy Warning]
features_test = features_df.iloc[len(train_df)::].copy()
class=class="string">"comment"># X_test
features_test.reset_index(inplace=True, drop=True)

class=class="string">"comment"># Restore the train // test split: slice
class="number">1 Series into class="number">2
labels_train = labels_df[:len(train_df)]
class=class="string">"comment"># y_train
labels_train.reset_index(inplace=True, drop=True)

labels_test = labels_df[len(train_df):]
class=class="string">"comment"># y_test
labels_test.reset_index(inplace=True, drop=True)

```

*

Next are standard steps for all datasets: `_scaling`, `classifiers`, `results_`

Scaling comes `_after_ test // train split`

```

In [21]: class=class="string">"comment"># scaling the Numeric columns
class=class="string">"comment"># StandardScaler range: -class="number">1
to class="number">1, MinMaxScaler range: zero to class="number">1

class=class="string">"comment"># class="keyword">from
sklearn.preprocessing class="keyword">import StandardScaler
class="keyword">from sklearn.preprocessing class="keyword">import
MinMaxScaler

class=class="string">"comment"># sklearn docs say
class=class="string">"comment"># class="string">"Don't cheat - fit only
on training data, then transform both"
class=class="string">"comment"># fit() expects 2D array: reshape(-
class="number">1, class="number">1) for single col or (class="number">1, -
class="number">1) single row

for i in numeri:
    arr = numpy.array(features_train[i])
    scale = MinMaxScaler().fit(arr.reshape(-class="number">1,
class="number">1))
    features_train[i] =
scale.transform(arr.reshape(len(arr),class="number">1))

    arr = numpy.array(features_test[i])
    features_test[i] =
scale.transform(arr.reshape(len(arr),class="number">1))

```

Classifier Selection

In [22]:

```
class=class="string">"comment"># prepare list
models = []

class=class="string">"comment">## -- Linear -- ##
class=class="string">"comment">#class="keyword">from sklearn.linear_model
class="keyword">import LogisticRegression
class=class="string">"comment">#models.append((class="string">"LogReg",Log
isticRegression()))
class=class="string">"comment">#class="keyword">from sklearn.linear_model
class="keyword">import SGDClassifier
class=class="string">"comment">#models.append((class="string">"StocGradDes
",SGDClassifier()))
class=class="string">"comment">#class="keyword">from
sklearn.discriminant_analysis class="keyword">import
LinearDiscriminantAnalysis
class=class="string">"comment">#models.append((class="string">"LinearDA",
LinearDiscriminantAnalysis()))
class=class="string">"comment">#class="keyword">from
sklearn.discriminant_analysis class="keyword">import
QuadraticDiscriminantAnalysis
class=class="string">"comment">#models.append((class="string">"QuadraticDA
", QuadraticDiscriminantAnalysis()))

class=class="string">"comment">## -- Support Vector -- ##
class=class="string">"comment">#class="keyword">from sklearn.svm
class="keyword">import SVC
class=class="string">"comment">#models.append((class="string">"SupportVect
orClf", SVC()))
class=class="string">"comment">#class="keyword">from sklearn.svm
class="keyword">import LinearSVC
class=class="string">"comment">#models.append((class="string">"LinearSVC",
LinearSVC()))
class=class="string">"comment">#class="keyword">from sklearn.linear_model
class="keyword">import RidgeClassifier
class=class="string">"comment">#models.append((class="string">"RidgeClf",R
idgeClassifier()))

class=class="string">"comment">## -- Non-linear -- ##
class="keyword">from sklearn.tree class="keyword">import
DecisionTreeClassifier
models.append((class="string">"DecisionTree",DecisionTreeClassifier()))
class="keyword">from sklearn.naive_bayes class="keyword">import GaussianNB
models.append((class="string">"GaussianNB",GaussianNB()))
class="keyword">from sklearn.neighbors class="keyword">import
KNeighborsClassifier
models.append((class="string">"K-Neighbors", KNeighborsClassifier()))

class=class="string">"comment">## -- Ensemble: bagging -- ##
class=class="string">"comment">#class="keyword">from sklearn.ensemble
class="keyword">import RandomForestClassifier
class=class="string">"comment">#models.append((class="string">"RandomFores
t", RandomForestClassifier()))
class=class="string">"comment">## -- Ensemble: boosting -- ##
class=class="string">"comment">#class="keyword">from sklearn.ensemble
class="keyword">import AdaBoostClassifier
class=class="string">"comment">#models.append((class="string">"AdaBoost",
AdaBoostClassifier()))
```

```

class=class="string">"comment">#class="keyword">from sklearn.ensemble
class="keyword">import GradientBoostingClassifier
class=class="string">"comment">#models.append((class="string">GradientBoo
st", GradientBoostingClassifier()))

class=class="string">"comment">## -- NeuralNet(simplest) -- ##
class=class="string">"comment">#class="keyword">from sklearn.linear_model
class="keyword">import Perceptron
class=class="string">"comment">#models.append((class="string">SingleLayer
Ptron",Perceptron()))
class=class="string">"comment">#class="keyword">from
sklearn.neural_network class="keyword">import MLPClassifier
class=class="string">"comment">#models.append((class="string">MultiLayerP
tron", MLPClassifier()))

print(models)

```

```

[('DecisionTree', DecisionTreeClassifier()), ('GaussianNB', GaussianNB()),
('K-NNneighbors', KNeighborsClassifier())]

```

compatibility block for pasting in from sample code

```

In [23]: class=class="string">"comment"># dataset names
X_train = features_train
y_train = labels_train
X_test = features_test
y_test = labels_test
labels_col = class="string">'label'
class=class="string">"comment"># library names
pd = pandas
np = numpy

```

Target Label Distributions (standard block)

```

In [24]: class=class="string">"comment"># class="keyword">from our local library
show_labels_dist(X_train,X_test,y_train,y_test)

```

```

features_train: 63280 rows, 121 columns
features_test: 22544 rows, 121 columns

```

```

labels_train: 63280 rows, 1 column
labels_test: 22544 rows, 1 column

```

Frequency and Distribution of labels

	label	%_train	label	%_test
label				
normal	33672	53.21	9711	43.08
attack	29608	46.79	12833	56.92

... this is a good place for the ClassBalance visualizer ...

Fit and Predict (standard block)

```

In [25]: class=class="string">"comment"># evaluate each model in turn
results = []

print(class="string">'macro average: unweighted mean per label')
print(class="string">'weighted average: support-weighted mean per label')
print(class="string">'MCC: correlation between prediction and ground
truth')
print(class="string">'      (+class="number">1 perfect, class="number">0
random prediction, -class="number">1 inverse)\n')

for name, clf in models:
    trs = time()
    print(class="string">'Confusion Matrix:', name)

    clf.fit(X_train, y_train)
    ygx = clf.predict(X_test)
    results.append((name, ygx))

    tre = time() - trs
    print (class="string">"Run Time {}
seconds".format(round(tre,class="number">2)) + class="string">'\\n')

class=class="string">"comment"># Easy way to ensure that the confusion
matrix rows and columns
class=class="string">"comment">#   are labeled exactly class="keyword">as
the classifier has coded the classes
class=class="string">"comment">#   [[note the _ at the end of clf.classes_
]]

    show_metrics(y_test, ygx, clf.classes_)
class=class="string">"comment"># class="keyword">from our local library
    print(class="string">'\\nParameters: ', clf.get_params(),
class="string">'\\n\\n')

```

```

macro average: unweighted mean per label
weighted average: support-weighted mean per label
MCC: correlation between prediction and ground truth
      (+1 perfect, 0 random prediction, -1 inverse)

```

```

Confusion Matrix: DecisionTree
Run Time 0.96 seconds

```

	pred:attack	pred:normal
train:attack	11094	1739
train:normal	423	9288

~~~~

```

macro avg : FPR = 0.090 FNR = 0.090
weighted avg : FPR = 0.096 FNR = 0.096

```

~~~~

	precision	recall	f1-score	support
attack	0.963	0.864	0.911	12833
normal	0.842	0.956	0.896	9711
accuracy			0.904	22544

macro avg	0.903	0.910	0.903	22544
weighted avg	0.911	0.904	0.905	22544

~~~~

MCC: Overall : 0.813

Parameters: {'ccp\_alpha': 0.0, 'class\_weight': None, 'criterion': 'gini', 'max\_depth': None, 'max\_features': None, 'max\_leaf\_nodes': None, 'min\_impurity\_decrease': 0.0, 'min\_samples\_leaf': 1, 'min\_samples\_split': 2, 'min\_weight\_fraction\_leaf': 0.0, 'monotonic\_cst': None, 'random\_state': None, 'splitter': 'best'}

Confusion Matrix: GaussianNB

Run Time 0.41 seconds

|              |             |             |
|--------------|-------------|-------------|
|              | pred:attack | pred:normal |
| train:attack | 3093        | 9740        |
| train:normal | 65          | 9646        |

~~~~

macro avg :	FPR = 0.383	FNR = 0.383
weighted avg :	FPR = 0.435	FNR = 0.435

~~~~

|              |           |        |          |         |
|--------------|-----------|--------|----------|---------|
|              | precision | recall | f1-score | support |
| attack       | 0.979     | 0.241  | 0.387    | 12833   |
| normal       | 0.498     | 0.993  | 0.663    | 9711    |
| accuracy     |           |        | 0.565    | 22544   |
| macro avg    | 0.738     | 0.617  | 0.525    | 22544   |
| weighted avg | 0.772     | 0.565  | 0.506    | 22544   |

~~~~

MCC: Overall : 0.334

Parameters: {'priors': None, 'var_smoothing': 1e-09}

Confusion Matrix: K-NNeighbors

Run Time 5.08 seconds

	pred:attack	pred:normal
train:attack	10293	2540
train:normal	832	8879

~~~~

|                |             |             |
|----------------|-------------|-------------|
| macro avg :    | FPR = 0.142 | FNR = 0.142 |
| weighted avg : | FPR = 0.150 | FNR = 0.150 |

~~~~

	precision	recall	f1-score	support
attack	0.925	0.802	0.859	12833
normal	0.778	0.914	0.840	9711
accuracy			0.850	22544

```

macro avg      0.851    0.858    0.850    22544
weighted avg   0.862    0.850    0.851    22544

~~~~

MCC: Overall : 0.710

Parameters: {'algorithm': 'auto', 'leaf_size': 30, 'metric': 'minkowski',
'metric_params': None, 'n_jobs': None, 'n_neighbors': 5, 'p': 2, 'weights':
'uniform'}

```

Bias - Variance Decomposition (standard block)

```

In [27]: class=class="string">"comment"># class="keyword">from our local library
class=class="string">"comment"># reduce(cross-validation) folds for faster
results
folds = class="number">20
for name, clf in models:
    print(class="string">'Bias // Variance Decomposition:', name)
    bias_var_metrics(X_train,X_test,y_train,y_test,clf,folds)

```

```

Bias // Variance Decomposition: DecisionTree
Average bias: 0.099
Average variance: 0.035
Average expected loss: 0.108 "Goodness": 0.892

Bias // Variance Decomposition: GaussianNB
Average bias: 0.434
Average variance: 0.003
Average expected loss: 0.434 "Goodness": 0.566

Bias // Variance Decomposition: K-NNNeighbors
Average bias: 0.153
Average variance: 0.015
Average expected loss: 0.152 "Goodness": 0.848

```

*

*

Baseline Model

>Select this block - Go to the Run menu - Run all Above

Then paste in blocks below from the other examples
and run them one at a time

*

*

Hyperparameter Tuning

General pattern:

baseline model

> 1. Classifier selection

2. Fit and Predict

3. Bias-Variance Tradeoff

optimised model

> 4. Select strategy and hyperparameters

5. Plug in the best parameter values

6. Fit and Predict

7. Bias-Variance Tradeoff

*

In [35]:

```
class=class="string">"comment">#
=====
class=class="string">"comment"># HYPERPARAMETER TUNING - NON-LINEAR
CLASSIFIERS
class=class="string">"comment">#
=====
class=class="string">"comment"># KEY IMPROVEMENTS:
class=class="string">"comment">#   class="number">1.
class_weight=class="string">'balanced' (handles class imbalance)
class=class="string">"comment">#   class="number">2. Multiple tuning
configurations tested
class=class="string">"comment">#   class="number">3. Bias-variance
analysis for tuned model
class=class="string">"comment">#
=====

class="keyword">import matplotlib.pyplot class="keyword">as plt
class="keyword">import seaborn class="keyword">as sns
class="keyword">from sklearn.metrics class="keyword">import
balanced_accuracy_score, f1_score, accuracy_score, matthews_corrcoef
class="keyword">from sklearn.metrics class="keyword">import
confusion_matrix
class="keyword">from sklearn.tree class="keyword">import
DecisionTreeClassifier
class="keyword">from sklearn.naive_bayes class="keyword">import GaussianNB
class="keyword">from sklearn.neighbors class="keyword">import
KNeighborsClassifier

print(class="string">"="*class="number">70)
print(class="string">"HYPERPARAMETER TUNING - NON-LINEAR CLASSIFIERS")
print(class="string">"="*class="number">70)
print(class="string">"\nClass Distribution(Note: Test set differs
class="keyword">from Train!):")
print(fclass="string">"  Train:
{(y_train=='normal').mean()*class="number">100:.1f}% normal,
{(y_train=='attack').mean()*class="number">100:.1f}% attack")
print(fclass="string">"  Test:
{(y_test=='normal').mean()*class="number">100:.1f}% normal,
{(y_test=='attack').mean()*class="number">100:.1f}% attack")
```

```
=====
HYPERPARAMETER TUNING - NON-LINEAR CLASSIFIERS
=====

Class Distribution (Note: Test set differs from Train!):
  Train: 53.2% normal, 46.8% attack
  Test:  43.1% normal, 56.9% attack
```

In [36]:

```
class=class="string">"comment">#
=====
class=class="string">"comment"># STEP class="number">1: BASELINE
COMPARISON - All Non-Linear Classifiers
class=class="string">"comment">#
=====

print(class="string">"\n" + class="string">"="*class="number">70)
print(class="string">"STEP class="number">1: BASELINE COMPARISON")
print(class="string">"="*class="number">70)

baseline_models = [
    (class="string">"DecisionTree",
    DecisionTreeClassifier(random_state=class="number">42)),
    (class="string">"GaussianNB", GaussianNB()),
    (class="string">"KNN", KNeighborsClassifier()),
]

all_results = {}
for name, clf in baseline_models:
    print(fclass="string">"\nTraining {name}...")
    trs = time()
    clf.fit(X_train, y_train)
    y_pred = clf.predict(X_test)
    tre = time() - trs

    all_results[name] = {
        class="string">'accuracy': accuracy_score(y_test, y_pred),
        class="string">'balanced_accuracy':
balanced_accuracy_score(y_test, y_pred),
        class="string">'f1_score': f1_score(y_test, y_pred,
average=class="string">'weighted'),
        class="string">'mcc': matthews_corrcoef(y_test, y_pred),
        class="string">'time': tre,
        class="string">'predictions': y_pred
    }
    print(fclass="string">" Bal_Acc: {all_results[name]
['balanced_accuracy']:.4f} | F1: {all_results[name]['f1_score']:.4f} |
Time: {tre:.2f}s")

class=class="string">"comment"># Identify best baseline
best_baseline = max(all_results.keys(), key=lambda x: all_results[x]
[class="string">'balanced_accuracy'])
baseline_ba = all_results[best_baseline]
[class="string">'balanced_accuracy']
print(fclass="string">"\n>>> BEST BASELINE: {best_baseline} (Bal_Acc:
{baseline_ba:.4f})")
```

```
=====
STEP 1: BASELINE COMPARISON
=====
```

Training DecisionTree...

Bal_Acc: 0.9112 | F1: 0.9054 | Time: 0.95s

Training GaussianNB...

Bal_Acc: 0.6172 | F1: 0.5058 | Time: 0.41s

Training KNN...

Bal_Acc: 0.8582 | F1: 0.8511 | Time: 4.90s

>>> BEST BASELINE: DecisionTree (Bal_Acc: 0.9112)

In [37]:

```
class=class="string">"comment">#
=====
class=class="string">"comment"># STEP class="number">2: HYPERPARAMETER
TUNING - DecisionTree
class=class="string">"comment">#
=====
class=class="string">"comment"># Tuning Strategy:
class=class="string">"comment"># class="number">1. max_depth: prevent
overfitting(None -> class="number">15, class="number">20)
class=class="string">"comment"># class="number">2. min_samples_split:
minimum samples to split
class=class="string">"comment"># class="number">3.
class_weight=class="string">'balanced': handles class imbalance
class=class="string">"comment">#
=====

print(class="string">"\n" + class="string">"="*class="number">70)
print(class="string">"STEP class="number">2: HYPERPARAMETER TUNING")
print(class="string">"="*class="number">70)

tuned_configs = [
    class=class="string">"comment"># Depth tuning
    (class="string">"DT_depth15",
    DecisionTreeClassifier(max_depth=class="number">15,
    random_state=class="number">42)),
    (class="string">"DT_depth20",
    DecisionTreeClassifier(max_depth=class="number">20,
    random_state=class="number">42)),
    class=class="string">"comment"># Class weight tuning(KEY IMPROVEMENT!)
    (class="string">"DT_balanced",
    DecisionTreeClassifier(class_weight=class="string">'balanced',
    random_state=class="number">42)),
    (class="string">"DT_depth15_balanced",
    DecisionTreeClassifier(max_depth=class="number">15,
    class_weight=class="string">'balanced', random_state=class="number">42)),
    (class="string">"DT_depth20_balanced",
    DecisionTreeClassifier(max_depth=class="number">20,
    class_weight=class="string">'balanced', random_state=class="number">42)),
    class=class="string">"comment"># Combined tuning
    (class="string">"DT_depth20_minsplit5",
    DecisionTreeClassifier(max_depth=class="number">20,
    min_samples_split=class="number">5, random_state=class="number">42)),
]

for name, clf in tuned_configs:
    print(fclass="string">"\nTraining {name}...")
    trs = time()
    clf.fit(X_train, y_train)
    y_pred = clf.predict(X_test)
    tre = time() - trs

    all_results[name] = {
        class="string">'accuracy': accuracy_score(y_test, y_pred),
        class="string">'balanced_accuracy':
balanced_accuracy_score(y_test, y_pred),
        class="string">'f1_score': f1_score(y_test, y_pred,
average=class="string">'weighted'),
```

```

        class="string">'mcc': matthews_corrcoef(y_test, y_pred),
        class="string">'time': tre,
        class="string">'predictions': y_pred
    }
    improvement = ((all_results[name][class="string">'balanced_accuracy']
- baseline_ba) / baseline_ba) * class="number">100
    print(fclass="string">" Bal_Acc: {all_results[name]
['balanced_accuracy']:.4f} | Improvement: {improvement:+.2f}%")

class=class="string">"comment"># Find best tuned model
tuned_names = [name for name, _ in tuned_configs]
best_tuned = max(tuned_names, key=lambda x: all_results[x]
[class="string">'balanced_accuracy'])
best_tuned_ba = all_results[best_tuned]
[class="string">'balanced_accuracy']
print(fclass="string">"\n>>> BEST TUNED: {best_tuned} (Bal_Acc:
{best_tuned_ba:.4f})")

```

```

=====
STEP 2: HYPERPARAMETER TUNING
=====

Training DT_depth15...
Bal_Acc: 0.9058 | Improvement: -0.59%

Training DT_depth20...
Bal_Acc: 0.9119 | Improvement: +0.07%

Training DT_balanced...
Bal_Acc: 0.9049 | Improvement: -0.69%

Training DT_depth15_balanced...
Bal_Acc: 0.9051 | Improvement: -0.66%

Training DT_depth20_balanced...
Bal_Acc: 0.9038 | Improvement: -0.81%

Training DT_depth20_minsplit5...
Bal_Acc: 0.9045 | Improvement: -0.74%

>>> BEST TUNED: DT_depth20 (Bal_Acc: 0.9119)

```

```
In [38]: class=class="string">"comment">#
=====
class=class="string">"comment"># STEP class="number">3: RESULTS COMPARISON
TABLE
class=class="string">"comment">#
=====

print(class="string">"\n" + class="string">"="*class="number">70)
print(class="string">"STEP class="number">3: ALL RESULTS COMPARISON")
print(class="string">"="*class="number">70)

class=class="string">"comment"># Create results DataFrame
results_df = pandas.DataFrame({
    name: {
        class="string">'Accuracy': r[class="string">'accuracy'],
        class="string">'Balanced_Acc':
r[class="string">'balanced_accuracy'],
        class="string">'F1_Score': r[class="string">'f1_score'],
        class="string">'MCC': r[class="string">'mcc'],
        class="string">'Time(s)': r[class="string">'time']
    } for name, r in all_results.items()
}).T

class=class="string">"comment"># Sort by balanced accuracy
results_df = results_df.sort_values(class="string">'Balanced_Acc',
ascending=False)
print(class="string">"\n" + results_df.to_string())

class=class="string">"comment"># Calculate improvement
overall_improvement = ((best_tuned_ba - baseline_ba) / baseline_ba) *
class="number">100
print(fclass="string">"\n>>> IMPROVEMENT: {baseline_ba:.4f} ->
{best_tuned_ba:.4f} ({overall_improvement:+.2f}%)"")
```

```
=====
STEP 3: ALL RESULTS COMPARISON
=====
```

	Accuracy	Balanced_Acc	F1_Score	MCC	Time(s)
DT_depth20	0.905651	0.911854	0.906092	0.816016	0.916306
DecisionTree	0.904941	0.911193	0.905387	0.814689	0.947382
DT_depth15	0.898820	0.905804	0.899305	0.803841	0.847328
DT_depth15_balanced	0.898066	0.905142	0.898555	0.802516	0.915583
DT_balanced	0.897889	0.904936	0.898378	0.802111	0.974737
DT_depth20_minsplit5	0.897179	0.904475	0.897673	0.801173	0.919364
DT_depth20_balanced	0.896513	0.903816	0.897011	0.799866	0.997346
KNN	0.850426	0.858198	0.851139	0.709554	4.904104
GaussianNB	0.565073	0.617163	0.505810	0.334323	0.407091

```
>>> IMPROVEMENT: 0.9112 -> 0.9119 (+0.07%)
```

In [39]:

```
class=class="string">"comment">#
=====
class=class="string">"comment"># VISUALIZATION class="number">1: Confusion
Matrix Comparison
class=class="string">"comment">#
=====

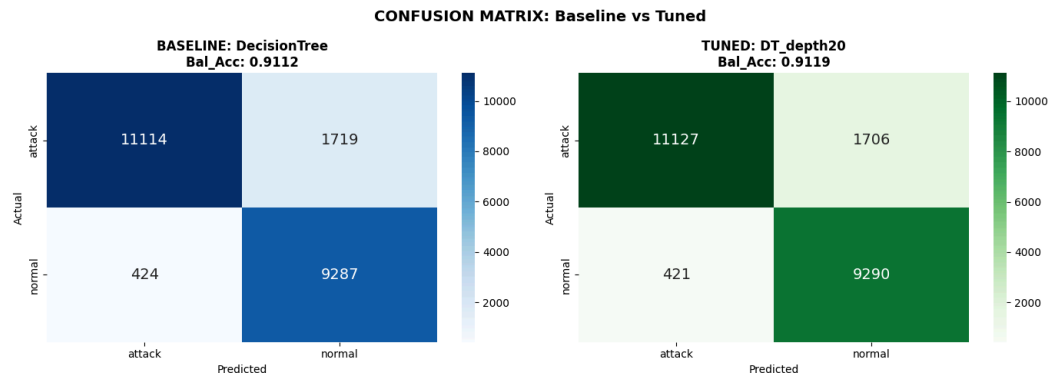
fig, axes = plt.subplots(class="number">1, class="number">2, figsize=
(class="number">14, class="number">5))

class=class="string">"comment"># Baseline DecisionTree confusion matrix
cm_baseline = confusion_matrix(y_test,
all_results[class="string">'DecisionTree'][class="string">'predictions'])
sns.heatmap(cm_baseline, annot=True, fmt=class="string">'d',
cmap=class="string">'Blues', ax=axes[class="number">0],
            xticklabels=[class="string">'attack',
class="string">'normal'], yticklabels=[class="string">'attack',
class="string">'normal'],
            annot_kws={class="string">'size': class="number">14})
axes[class="number">0].set_title(fclass="string">'BASELINE:
DecisionTree\nBal_Acc: {all_results["DecisionTree"]
["balanced_accuracy"]:.4f}',
            fontweight=class="string">'bold',
            fontsize=class="number">12)
axes[class="number">0].set_xlabel(class="string">'Predicted')
axes[class="number">0].set_ylabel(class="string">'Actual')

class=class="string">"comment"># Best Tuned confusion matrix
cm_tuned = confusion_matrix(y_test, all_results[best_tuned]
[class="string">'predictions'])
sns.heatmap(cm_tuned, annot=True, fmt=class="string">'d',
cmap=class="string">'Greens', ax=axes[class="number">1],
            xticklabels=[class="string">'attack',
class="string">'normal'], yticklabels=[class="string">'attack',
class="string">'normal'],
            annot_kws={class="string">'size': class="number">14})
axes[class="number">1].set_title(fclass="string">'TUNED:
{best_tuned}\nBal_Acc: {best_tuned_ba:.4f}',
            fontweight=class="string">'bold',
            fontsize=class="number">12)
axes[class="number">1].set_xlabel(class="string">'Predicted')
axes[class="number">1].set_ylabel(class="string">'Actual')

plt.suptitle(class="string">'CONFUSION MATRIX: Baseline vs Tuned',
            fontsize=class="number">14, fontweight=class="string">'bold')
plt.tight_layout()
plt.show()
```

Out[39]:



In [40]:

```
class=class="string">"comment">#
=====
class=class="string">"comment"># VISUALIZATION class="number">2: All
Models Comparison Bar Chart
class=class="string">"comment">#
=====

fig, ax = plt.subplots(figsize=(class="number">12, class="number">6))

class=class="string">"comment"># Sort by balanced accuracy
plot_df = results_df.sort_values(class="string">'Balanced_Acc',
ascending=True)

class=class="string">"comment"># Color best model green, baseline blue,
others light blue
colors = []
for name in plot_df.index:
    if name == best_tuned:
        colors.append(class="string">'class="comment">#2ecc71') # Green
for best
    elif name in [class="string">'DecisionTree',
class="string">'GaussianNB', class="string">'KNN']:
        colors.append(class="string">'class="comment">#3498db') # Blue
for baselines
    else:
        colors.append(class="string">'class="comment">#85c1e9') # Light
blue for other tuned

bars = ax.barh(plot_df.index, plot_df[class="string">'Balanced_Acc'],
color=colors, edgecolor=class="string">'black')

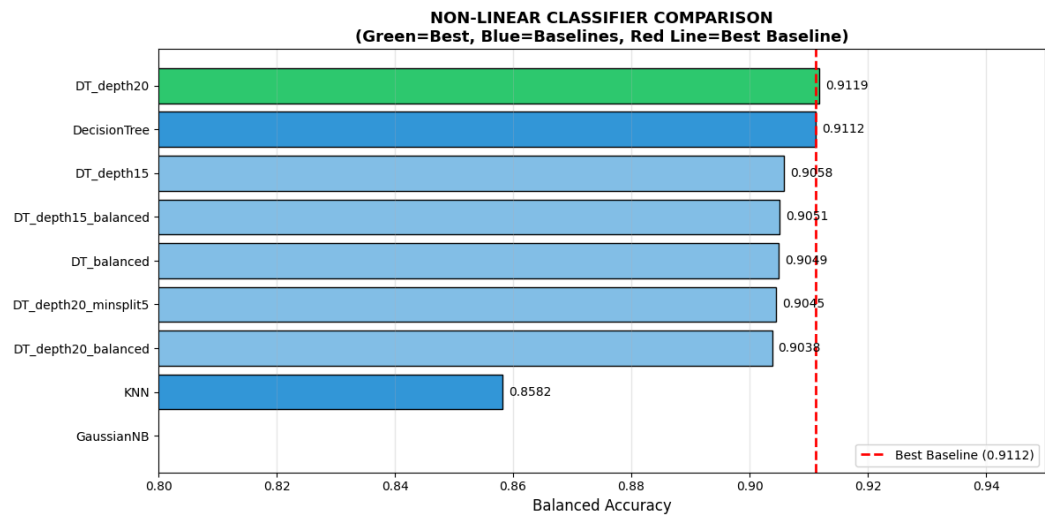
class=class="string">"comment"># Add baseline reference line
ax.axvline(x=baseline_ba, color=class="string">'red',
linestyle=class="string">'--', linewidth=class="number">2,
label=fclass="string">'Best Baseline({baseline_ba:.4f})')

ax.set_xlabel(class="string">'Balanced Accuracy',
fontsize=class="number">12)
ax.set_title(class="string">'NON-LINEAR CLASSIFIER
COMPARISON\n(Green=Best, Blue=Baselines, Red Line=Best Baseline)',
fontweight=class="string">'bold', fontsize=class="number">13)
ax.set_xlim(class="number">0.80, class="number">0.95)
ax.legend(loc=class="string">'lower right')
ax.grid(axis=class="string">'x', alpha=class="number">0.3)

class=class="string">"comment"># Add value labels
for bar, val in zip(bars, plot_df[class="string">'Balanced_Acc']):
    ax.annotate(fclass="string">'{val:.4f}', xy=(val, bar.get_y() +
bar.get_height()/class="number">2),
xytext=(class="number">5, class="number">0),
textcoords=class="string">'offset points', va=class="string">'center',
fontsize=class="number">10)

plt.tight_layout()
plt.show()
```

Out[40]:



In [41]:

```
class=class="string">"comment">#
=====

class=class="string">"comment"># STEP class="number">4: BIAS-VARIANCE
DECOMPOSITION FOR BEST MODEL
class=class="string">"comment">#
=====

print(class="string">"="*class="number">70)
print(class="string">"STEP class="number">4: BIAS-VARIANCE ANALYSIS - Best
Tuned Model")
print(class="string">"="*class="number">70)

class=class="string">"comment"># Recreate best model for bias-variance
analysis
if class="string">'balanced' in best_tuned:
    if class="string">'depth15' in best_tuned:
        best_clf = DecisionTreeClassifier(max_depth=class="number">15,
class_weight=class="string">'balanced', random_state=class="number">42)
    elif class="string">'depth20' in best_tuned:
        best_clf = DecisionTreeClassifier(max_depth=class="number">20,
class_weight=class="string">'balanced', random_state=class="number">42)
    else:
        best_clf =
DecisionTreeClassifier(class_weight=class="string">'balanced',
random_state=class="number">42)
elif class="string">'depth15' in best_tuned:
    best_clf = DecisionTreeClassifier(max_depth=class="number">15,
random_state=class="number">42)
elif class="string">'depth20' in best_tuned:
    best_clf = DecisionTreeClassifier(max_depth=class="number">20,
random_state=class="number">42)
else:
    best_clf = DecisionTreeClassifier(random_state=class="number">42)

print(fclass="string">"\nModel: {best_tuned}")
bias_var_metrics(X_train, X_test, y_train, y_test, best_clf,
folds=class="number">20)

class=class="string">"comment">#
=====

class=class="string">"comment"># FINAL CONCLUSION
class=class="string">"comment">#
=====

print(class="string">"\n" + class="string">"="*class="number">70)
print(class="string">"FINAL CONCLUSION")
print(class="string">"="*class="number">70)

b = all_results[class="string">'DecisionTree']
t = all_results[best_tuned]

print(fclass="string">"""
HYPERPARAMETER TUNING RESULTS:
=====

BASELINE(DecisionTree, default):
- Accuracy: {b[class="string">'accuracy']:.4f}
```

```
- Balanced Accuracy: {b[class="string">'balanced_accuracy']:.4f}
- F1-Score:          {b[class="string">'f1_score']:.4f}
- MCC:              {b[class="string">'mcc']:.4f}
```

BEST TUNED({best_tuned}):

```
- Accuracy:          {t[class="string">'accuracy']:.4f}
- Balanced Accuracy: {t[class="string">'balanced_accuracy']:.4f}
- F1-Score:          {t[class="string">'f1_score']:.4f}
- MCC:              {t[class="string">'mcc']:.4f}
```

IMPROVEMENT: {overall_improvement:+.2f}%

KEY TUNING STRATEGIES:

```
class="number">1. max_depth: Controls tree depth to prevent overfitting
class="number">2. class_weight=class="string">'balanced': Handles
imbalanced class distribution
class="number">3. min_samples_split: Minimum samples required to split
```

WHY DECISIONTREE IS BEST FOR NON-LINEAR:

```
- Fast training(no distance calculations like KNN)
- Handles high-dimensional data well(class="number">121 features)
- Captures complex decision boundaries for attack patterns
- Interpretable model(can visualize feature importance)
```

```
class="string">""")
```

```
print(class="string">"="*class="number">70)
```

```
=====
STEP 4: BIAS-VARIANCE ANALYSIS - Best Tuned Model
=====
```

Model: DT_depth20

Average bias: 0.088

Average variance: 0.034

Average expected loss: 0.104 "Goodness": 0.896

```
=====
FINAL CONCLUSION
=====
```

HYPERPARAMETER TUNING RESULTS:

```
=====
```

BASELINE (DecisionTree, default):

```
- Accuracy:          0.9049
- Balanced Accuracy: 0.9112
- F1-Score:          0.9054
- MCC:              0.8147
```

BEST TUNED (DT_depth20):

```
- Accuracy:          0.9057
- Balanced Accuracy: 0.9119
- F1-Score:          0.9061
- MCC:              0.8160
```

IMPROVEMENT: +0.07%

KEY TUNING STRATEGIES:

1. `max_depth`: Controls tree depth to prevent overfitting
2. `class_weight='balanced'`: Handles imbalanced class distribution
3. `min_samples_split`: Minimum samples required to split

WHY DECISIONTREE IS BEST FOR NON-LINEAR:

- Fast training (no distance calculations like KNN)
- Handles high-dimensional data well (121 features)
- Captures complex decision boundaries for attack patterns
- Interpretable model (can visualize feature importance)

=====